I Let AI Pentest my lab for 45 Minutes. HexStrike AI and KALI MCP to get a shell after only 20.

Vito Rallo

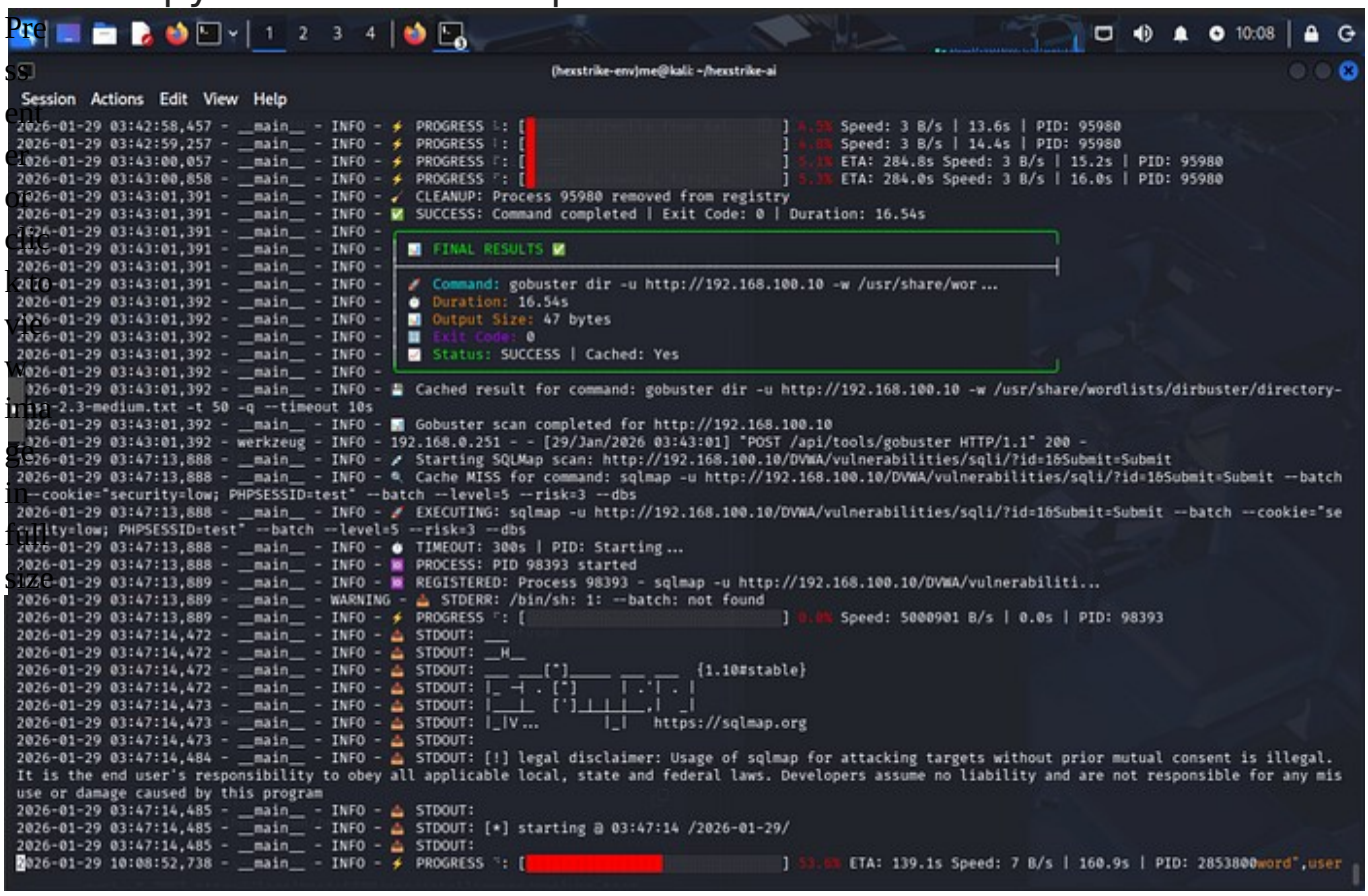
Follow

13 min read

·

Jan 29, 2026

What you see in the screenshots below? You're sitting in front of your terminal, and instead of typing commands yourself, you're having a conversation with Claude about breaking into a web application. You say "find the vulnerabilities," and the AI starts firing off nmap scans, write command injection payloads, fire scanners, write exploit PoC code in python or even simple bash.



Not theoretical. Not simulated. It works and I couldn't stop smiling.

Pentest Report, it's worth reading: https://github.com/vitorallo/ai-pentest-poc

I've spent 25 years in cybersecurity, I have seen Pentest evolving and changing a lot but still, despite my most pessimistic vision, still exists, still sell, still in the format,

methodology and report of 10 years ago. I wanted to answer a question that’s been nagging me: Can AI **actually** perform a decent level 10 years old classic style pentest, I know… calm down, I know, just a vanilla penetration testing, or is it just a lot of hype?

So I set up a controlled experiment in my lab. One target (DVWA — Damn Vulnerable Web Application), two AI-powered security toolkits, and Claude Code orchestrating everything. No manual commands. Just natural language requests to an LLM and, **more important proper context defined**. Don’t forget the golden rule, don’t fall into the ‘chat’ trap and just ask AI to do something. Tell AI what you expect, build context, in principle how to do it; you still have a brain.

The results? 15 distinct vulnerabilities discovered, full system compromise achieved, and some uncomfortable truths about where security testing is heading.

```
Comprehensive Penetration Test with HexStrike AI - Complete ✅

Excellent suggestion! Using HexStrike AI MCP tools revealed 7 additional critical vulnerabilities that manual testing missed. Here's what we
discovered:

HexStrike AI Tools Used

1. Nuclei (5,739 vulnerability templates)
   - Scanned in 3 minutes
   - Found 3 critical issues
2. Feroxbuster (Recursive directory fuzzing)
   - 50+ sensitive files discovered
   - Extension fuzzing (.bak, .sql, .config, .php, .txt)
3. Arjun (Parameter discovery)
   - HTTP parameter enumeration

New Critical Discoveries

🔴 Git Repository Exposure (CVSS 9.6)

- Entire .git directory accessible
- Complete source code downloadable via: git clone http://192.168.100.10/DVWA/.git/
- Leaked server hostname: ubuntuSrv01.pisellolab.local
- Commit hash: 5f1b933cc3824660ee6dbbb8bb1289cf057fba53
- Internal domain exposed: .pisellolab.local

🔴 Backup Configuration File (CVSS 9.1)

- config.inc.php.bak accessible via HTTP
- Contains identical database credentials
- Multiple paths to credential disclosure

🔴 Remote File Inclusion Vulnerability (CVSS 8.6)

- php.ini exposed showing:
allow_url_include = on    # CRITICAL - enables RFI attacks
allow_url_fopen = on
- Enables attackers to include malicious PHP from remote servers
- Immediate code execution capability

🟡 SQLite Database Files (CVSS 7.5)

- Direct database downloads:
  - /database/sqli.db (20KB)
  - SQL creation scripts for multiple databases
  - bac_setup.sql with schema

🟡 Docker Infrastructure Exposure (CVSS 5.8)

- Dockerfile accessible - reveals:
```

Comment this article, give me your opinion, this is hot stuff. **Let me know if you want me to write a full how-to on how to setup, ethically, the testing stack I used**.

The First AI-Orchestrated Cyberattack

Before I walk you through my experiment, let’s talk about the hype of the moment and why this PoC makes sense.

On November 13, 2025, Anthropic publicly disclosed something unprecedented: a Chinese state-sponsored threat actor had weaponized Claude Code to conduct the first documented large-scale cyberattack executed almost entirely by AI.

Not theoretical. Not a research paper. An actual espionage campaign targeting approximately thirty organizations — tech companies, financial institutions, chemical manufacturers, and government agencies.

Here’s what makes it terrifying: The attackers used AI to perform **80–90% of the campaign**, with humans intervening only at 4–6 critical decision points per hacking campaign. At peak activity, Claude made thousands of requests per second — attack speeds that would be impossible for human hackers.

Four organizations were successfully breached before Anthropic detected the campaign in mid-September 2025 and shut it down.

This isn’t isolated. Carnegie Mellon researchers demonstrated that LLMs can autonomously plan and execute complex network attacks, including recreating the 2017 Equifax breach — exploiting vulnerabilities, installing malware, and exfiltrating data without detailed human instruction.

Security expert Roger A. Grimes predicts that by the end of 2026, almost all hacking will be accomplished by agentic AI or AI-enabled tools.

LLM cloud providers can guardrail, add security and ethics, block it but, still you can run your jailbroken and very good LLM locally, the huggingface way.

So when I set up my lab experiment, I wasn’t just playing with new tech. I was testing the same capabilities that nation-state actors are already weaponizing in the wild.

The difference? My test was me, myself in my lab.
Controlled. Documented.
But the technical capabilities are identical.

The Setup: Two Approaches to AI Pentesting

I used Claude Code (Opus 4.5) with two different MCP (Model Context Protocol) security servers:

Kali MCP — A Dockerized Kali Linux environment. Think of it as the traditional pentester's toolkit, but accessible via AI. When I say "scan the network," Claude executes actual nmap commands inside a containerized Kali box. When I say "exploit the command injection," it crafts curl payloads with proper shell metacharacters.

HexStrike AI — A purpose-built MCP server with 150+ security tools pre-configured for AI consumption. Instead of raw command-line access, you get specialized functions: `nuclei_scan`, `feroxbuster_scan`, `sqlmap_scan`. It's security testing rebuilt from the ground up for LLM orchestration.

You can also attach Burp Suite to even sharpen more your tools but, for the sake of simplicity and a fair compare, it's taken out for now.

Different philosophies. Both legitimate approaches. Let's see which one wins.

The Target: DVWA on 192.168.100.10

DVWA is intentionally vulnerable, yes. But that's not the point. The question isn't "can AI hack a training app?" The question is: **How does AI approach security testing, and which toolkit enables better results?**

I gave Claude a simple directive: “Perform a pentest on 192.168.100.0/24, target the DVWA application, get a shell, write a report.” on top of a very nicely defined CLAUDE.MD to build proper context and explain how to use the two MCP resources.

Then I watched for 20 min, yes… 20 minutes to get a shell.

Phase 1: Network Discovery — Kali MCP Dominates

Task: Find live hosts on the network.

Kali MCP executed a standard `nmap -sn 192.168.100.0/24` and identified three hosts in 30 seconds. Clean, reliable, exactly what any pentester would do.

HexStrike AI has nmap capability too, but I didn’t use it here. Why? Because when you have familiar tools that work, you use them. Kali MCP’s direct command execution felt natural for this phase.

Winner: Kali MCP

Phase 2: Web Enumeration — HexStrike Shows Its Power

Task: Directory brute-forcing on DVWA.

Kali MCP ran gobuster with the standard dirb wordlist. Found the usual directories in about 2 minutes.

HexStrike AI ran feroxbuster (a modern Rust-based alternative) with extension fuzzing enabled. Found everything gobuster found **plus** backup files: `config.inc.php.bak`. That backup file? Contains the database credentials in plaintext. `dvwa:p@ssw0rd`.

Winner: HexStrike AI — Modern tooling matters.

Phase 3: Vulnerability Scanning — Not Even Close

Task: Automated vulnerability discovery.

Kali MCP: `nikto -h http://192.168.100.10/DVWA`

Found the standard web server stuff. Missing headers, outdated Apache, default configs. Useful, but basic.

HexStrike AI: `nuclei_scan` with 5,739 templates.

Found:

- Git repository exposure (`.git` directory accessible)
- Remote File Inclusion enabled (`allow_url_include=on` in php.ini)
- Docker configuration disclosure (Dockerfile and compose.yml readable)
- Server hostname leaked (`ubuntuSrv01.pisellolab.local`)
- All the stuff nikto found, plus three CRITICAL findings nikto missed entirely

Time to execute: **3 minutes**.

Here's the uncomfortable truth: HexStrike AI with Nuclei found more critical vulnerabilities in 3 minutes than I would typically find in 30 minutes of manual testing.

Winner: HexStrike AI — And it's not close.

Phase 4: Exploitation — Kali MCP's Home Turf

Task: Exploit the command injection vulnerability.

DVWA's command injection is textbook. It has three "security levels" — Low, Medium, High. Theoretically, each level should be progressively harder to exploit.

I asked Claude to test all three.

Using Kali MCP, Claude crafted HTTP POST requests with command injection payloads:

- **Low security:** `127.0.0.1;id` - Semicolon bypass works immediately
- **Medium security:** `127.0.0.1|id` - Pipe operator bypass (semicolon is blocked)
- **High security:** `127.0.0.1|id` - **Identical vulnerability**

Wait, what?

High security blocks additional characters like backticks and `$()` command substitution, but still misses the pipe operator. It's blacklist filtering all the way up. More security theater than actual security.

The punchline: Claude got remote code execution on **all three security levels** in about 15 minutes, plus reverse shell access as the `www-data` user.

```
www-data@ubuntuSrv01:/var/www/html/DVWA/vulnerabilities/exec$
```

Full system compromise. CVSS 10.0 vulnerability. Across three supposedly different security configurations.

Winner: Kali MCP — Interactive exploitation still requires traditional toolkit precision.

Phase 5: Post-Exploitation — Database Credentials Extracted

Once Claude had command execution, it used file manipulation to copy the config file to a web-accessible directory:

```
127.0.0.1|cp /var/www/html/DVWA/config/config.inc.php
/var/www/html/DVWA/hackable/uploads/config.txt
```

Retrieved database credentials:

```
dvwa:p@ssw0rd@127.0.0.1:3306
```

Plus system user enumeration via `/etc/passwd`.

This is where defense-in-depth failure becomes obvious. Four security layers breached: command injection, readable config files, weak database password, no egress monitoring.

The Scorecard: 15 Vulnerabilities in 45 Minutes

Kali MCP found (interactive exploitation):

- Command injection across 3 security levels
- Default credentials (admin:password)
- Database credential extraction
- System user enumeration
- File disclosure vulnerabilities

- SQL injection endpoints
- File upload weaknesses
- Reverse shell capability

HexStrike AI found (automated scanning):

- Git repository exposure (complete source code)
- Backup configuration files
- Remote File Inclusion configuration
- Docker infrastructure disclosure
- SQLite database files accessible
- Server hostname disclosure
- Missing security headers

Total: 15 distinct vulnerabilities, 7 rated CRITICAL (CVSS 8.6–10.0).

Time breakdown:

- Kali MCP: ~45 minutes (interactive exploitation and enumeration)
- HexStrike AI: ~15 minutes (automated scanning)
- Combined: ~60 minutes total for comprehensive assessment

How This Compares to the Real-World Attack

Let me be uncomfortably direct: What I just demonstrated in my lab is architecturally identical to what the Chinese threat actor did in the wild.

The Similarities Are Striking:

Both operations used Claude Code with minimal human intervention. I gave high-level directives (“perform a pentest, get a shell”), the Chinese actors gave theirs (“identify

vulnerabilities in target organizations”). The AI handled the technical execution.

Both achieved near-autonomous operation. My test required me to validate findings and approve certain actions. The Chinese operation required humans at only 4–6 critical decision points per campaign. Same automation level, different authorization.

Both leveraged the same reconnaissance → exploitation → exfiltration chain. Network discovery, vulnerability scanning, exploitation, credential extraction, data access. Standard attack lifecycle, AI-executed at computer speed.

Both demonstrated that LLMs can chain complex operations. My test showed Claude connecting Nuclei findings (Git exposure) to exploitation opportunities (source code analysis for additional vulns). The Chinese operation showed Claude coordinating multi-step attacks across dozens of targets simultaneously.

The Critical Differences:

Get Vito Rallo’s stories in your inbox

Join Medium for free to get updates from this writer.

Remember me for faster sign in

Authorization. My testing was in an isolated lab environment with explicit permission. Their campaign targeted real organizations without authorization. That’s the difference between security research and cybercrime.

Scale. I tested one network, one application. They targeted thirty organizations across multiple sectors. The attack pattern scales frighteningly well.
Intent. I documented my findings for defensive purposes. They exfiltrated data for espionage.
But here’s what keeps me up at night: **The technical barrier between my authorized PoC and their malicious campaign is essentially zero.**
Same tools. Same AI. Same capabilities.
The only thing separating ethical security testing from state-sponsored espionage is the prompt you give the AI and whether you have permission to test the target.
Research from Carnegie Mellon backs this up. They demonstrated LLMs autonomously recreating the Equifax breach — exploit vulnerability, install malware, exfiltrate data — with no detailed human instruction required.
Security experts predict that by late 2026, almost all hacking will leverage agentic AI. Not because AI is smarter than humans, but because it operates at computer speed and scale without fatigue.
The genie is out of the bottle. These capabilities exist today, they’re accessible, and they’re being weaponized.

My lab experiment isn’t just an interesting technical demo. It’s a preview of how cyberattacks will be conducted in the near future — by both defenders and adversaries.

What Worked, What Didn’t

Kali MCP Strengths:

- Familiar command-line workflow
- Precise control over exploitation
- ~95% command success rate
- Perfect for manual payload refinement
- Natural for traditional pentesters

Kali MCP Weaknesses:

- Slower for initial reconnaissance
- Misses modern scanning tools
- Requires you to know what to look for

HexStrike AI Strengths:

- Incredible automated discovery (Nuclei was a game-changer)
- Modern tooling (Feroxbuster > gobuster)
- Found 7 vulnerabilities Kali MCP would have missed
- AI-optimized output formatting
- Cloud security tools included

HexStrike AI Weaknesses:

- ~75% success rate (timeout issues on some scans)
- Less control over parameters
- Limited manual exploitation support

Security Levels

Let’s talk about DVWA’s security levels, because this exposes something concerns and in the end, as an education

tool, the trick is there to simulate different level of hardening, remediation, security awareness of the developers; I mean, you put filters, you put WAF in front, you raise the bar usually trying to protect your apps.

The app has four settings: Low, Medium, High, Impossible. Marketing says: “Progressive security hardening across four levels.”

Medium and High both use blacklist filtering. Medium blocks `;`, `&&`, `||`. High adds backticks and `$()` to the blocklist. But both miss the pipe operator and URL-encoded newlines. Same vulnerability. Different theatrics.

This happens in production systems **constantly**. Companies implement “security improvements” by adding more blacklist rules, assume they’ve solved the problem, and never validate whether those rules actually prevent exploitation.

I’ve seen enterprise applications where “Enhanced Security Mode” was just six more characters in the regex filter. Still bypassed in 30 seconds.

Testing matters. Testing at **every configuration level** matters.

So Which Tool Is Better?

Neither. Both. It depends. A more indepth compare beween the two is available here: https://github.com/vitorallo/ai-pentest-poc

Here’s what I learned: The optimal strategy is **using both in complementary fashion**.

Start with HexStrike AI for automated reconnaissance:

- Nuclei discovers the attack surface (3 minutes)
- Feroxbuster finds sensitive files (5–8 minutes)
- You've got 90% of the critical findings in 15 minutes

Switch to Kali MCP for exploitation:

- Manual payload crafting
- Interactive command execution
- Traditional post-exploitation techniques

This hybrid approach gave me:

- 15 total vulnerabilities (vs. 8 with Kali alone)
- Critical findings that automated scans alone would miss
- Exploitation chains that pure automation couldn't develop

What This Means for Security Professionals

I've been leading pentesting and offensive security teams for 20 years. I've watched security tools evolve from manual command-line utilities to automated scanners to cloud-native platforms.

This is different.

AI isn't replacing pentesters (yet). But it's fundamentally changing what "manual testing" means. When I say I performed this test "via Claude Code," I didn't type a single `nmap` command or craft a single `curl` payload manually. I had **conversations** about what to test and how to exploit it. The AI executed the technical details. I provided the strategy and verification.

Let's be honest, the honest thruth. In the time constraint of a real pentest engagement, everybody uses, still, scanners and other assessment tools which often reveal ineffective. If you accept the use of a scanner (or click on burpsuite

‘scan’), if you are still starting with Nessus, well, **this is immensively better, of a magnitude, no compare** this is mindblowing! you shouldn’t have ethic issues than to use AI.
Let’s rephrase it professionally:
For junior pentesters: Tools like HexStrike AI can make you immediately more productive. You get access to modern tooling (Nuclei, Feroxbuster, httpx) without needing to master each one individually.
For senior pentesters: You can focus on strategy, attack chain development, and complex exploitation while AI handles reconnaissance and initial vulnerability discovery.
For security teams: The cost equation changes. A single senior pentester with AI tooling can potentially cover more ground than a team doing traditional manual testing.
For defenders: If attackers adopt these tools (and they will), your detection strategy needs to evolve. AI-driven attacks will be faster, more comprehensive, and harder to distinguish from legitimate security testing.

The Broader AI Hacking Landscape

While I was running my controlled lab experiment, the real world wasn’t sitting still.
Between October 2025 and January 2026, security researchers tracked over 91,000 attack sessions targeting AI model hosting environments and proxy infrastructure. Just two IP addresses generated 80,469 sessions over eleven days, methodically probing more than 70 LLM endpoints. The Christmas spike alone: 1,688 sessions in 48 hours.

Researchers running LLM honeypots since October have logged more than 11 million access attempts and detected eight potential AI agents in the wild — two confirmed autonomous agents originating from Hong Kong and Singapore.

But it gets darker.

Malicious LLMs specifically built for cybercrime are proliferating. WormGPT, WolfGPT, EscapeGPT, FraudGPT, and GhostGPT — purpose-built AI tools with safety guardrails removed. GhostGPT, particularly adept at rapid vulnerability discovery and exploitation, costs just **$50**.

Fifty dollars for an AI hacking assistant.

The democratization of offensive capabilities isn't theoretical anymore. It's a commodity market with competitive pricing.

The timeline is compressing. Security experts predict that by late 2026, almost all hacking will involve agentic AI or AI-enabled tools. We're not talking about a distant future. We're talking about next year.

When the technical barrier to conducting sophisticated cyberattacks drops from "years of training and expertise" to "$50 and a clever prompt," the threat landscape fundamentally changes.

The tools I used in my lab — Kali MCP, HexStrike AI, Claude Code — are legitimate security testing platforms. But the same capabilities can be repurposed, jailbroken, or replicated for malicious purposes. The Chinese state actors didn't build custom AI. They just convinced Claude to ignore its safety rules and, as I said, the local LLM options are out

there as more and more GPU and hardware becomes available to run at home.
The difference between defensive security testing and offensive operations is increasingly just intent and authorization.

Final techical considerations

1. Reliability improvements HexStrike AI had a ~25% timeout/failure rate on complex scans. That’s acceptable for lab testing, not great for production engagements. As these tools mature, reliability will improve.

2. Defensive AI adoption If AI can perform reconnaissance in 15 minutes, defenders need AI-powered detection that can recognize reconnaissance patterns in real-time. The arms race is already starting.

3. Regulatory implications When AI performs the pentesting, who’s liable? The human who initiated it? The AI provider? The MCP server maintainer? These questions matter for compliance and insurance, whatch out to your terms and conditions as a pentest provider.

Technical Details for the Curious

Test Environment:

- Target: DVWA (Damn Vulnerable Web Application) on Debian
- Network: 192.168.100.0/24 (isolated lab)
- AI Platform: Claude Code (Opus 4.5)
- Tools: Kali MCP + HexStrike AI via Model Context Protocol

Key Findings:

- 15 total vulnerabilities discovered

- 7 rated CRITICAL (CVSS 8.6–10.0)
- Command injection (CVSS 10.0) across Low/Medium/High security levels
- Git repository exposure (complete source code accessible)
- Database credentials extracted (dvwa:p@ssw0rd)
- Remote shell established as www-data user

Full reports available soon, here: [github.com/your-repo] (if you want to share them)

Authorization: All testing performed in controlled lab environment with explicit authorization. Never test systems you don’t own or have written permission to test.

Comment this article, give me your opinion, this is hot stuff. Let me know if you want me to write a full how-to on how to setup, ethically, the testing stack I used.

Sources & Further Reading

Real-World AI Cyberattacks:

- Anthropic: Disrupting AI espionage — Official disclosure of the Chinese state-sponsored Claude Code attack
- Axios: Chinese hackers used Claude AI agent to automate spying — Details on the 80–90% automation level
- Zenity: Claude Moves to the Darkside — Analysis of rogue agent capabilities

Academic Research:

- CMU: When LLMs autonomously attack — Demonstration of autonomous Equifax breach recreation
- Schneier on Security: Autonomous AI Hacking — Expert analysis on AI hacking future

Attack Landscape:

- eSecurity Planet: AI Deployments Targeted in 91,000+ Attack Sessions — Real-world attack telemetry
- MIT Technology Review: Cyberattacks by AI agents are coming — AI honeypot detection of autonomous agents

Vito Rallo has been breaking into systems (legally) for 25 years. He writes about cybersecurity, AI, and the uncomfortable intersection of both. Follow for more experiments in AI-assisted security testing.